# Requirements & Environment

Kubernetes Production · Lesson 2

What to prepare **before** running Kubespray

`kubespray.io` · `kubernetes.io/docs`

# Learning Objectives

By the end of this lesson you will be able to:

- Distinguish the **Ansible control node** from the **target nodes**
- Provision a control node: **Python**, **ansible-core**, **netaddr**, a **venv**
- Meet target-node prerequisites: **OS**, **CPU/RAM**, **swap off**, **IP forwarding**
- Load the right **kernel modules** (`overlay`, `br_netfilter`) and **sysctls**
- Ensure each node has a **unique** hostname, MAC and product_uuid
- Open the **required ports** for the control plane, workers and the **CNI**

> Reference: Kubespray *Getting started* + *port/kernel requirements*, and the kubeadm *host prerequisites* — these are merged here into one prep checklist.

# 2.1 Control Node vs Target Nodes

# Two Sides to Every Deployment

A Kubespray install always has **two distinct sides**:

| Side | Role |
|---|---|
| **Control node** | Your workstation / bastion that **runs Ansible**. It is **not** part of the cluster — it only needs Python + SSH to every target |
| **Target nodes** | The future **control-plane**, **etcd** and **worker** machines that become the cluster |

- Kubespray drives the targets with **Ansible over SSH** — it performs the same host prep a manual `kubeadm` install needs (swap off, modules, sysctls, ports).
- Most of that prep is **automated** by Kubespray, with one big exception…

> **Firewalls are not managed** by Kubespray — you open the ports yourself.
> Everything in this lesson is what you put in place **before running Kubespray**.

# 2.2 The Ansible Control Node

# Ansible Control-Node Requirements

The control node runs Ansible against the cluster; it needs only Python, the right Ansible, and `netaddr`.

| Requirement | Value (per docs) |
|---|---|
| Ansible | **v2.14+** (README); current tree pins **ansible-core ≥2.18.0, <2.19.0** |
| Python (control node) | **3.11–3.13** for Ansible 2.18.x |
| Jinja | **2.11+** |
| Python module | **`netaddr`** ( `python-netaddr` ) on the machine running Ansible |
| Install method | Python **venv** + `pip install -r requirements.txt` |

> Using `requirements.txt` pins the **exact** ansible-core Kubespray expects and pulls in Jinja and `netaddr` — don't install Ansible from your distro packages.

# Install Ansible in a venv

Create an isolated environment and install exactly what Kubespray pins:

```
VENVDIR=kubespray-venv
KUBESPRAYDIR=kubespray
python3 -m venv $VENVDIR
source $VENVDIR/bin/activate
cd $KUBESPRAYDIR
pip install -r requirements.txt   # Ansible 2.18.x, jinja, netaddr
```

Verify before going further:

```
ansible --version    # ansible-core within 2.18.x
python3 --version    # 3.11 - 3.13 for Ansible 2.18.x
```

> A venv keeps Kubespray's pinned Ansible away from system Python — re-`source` the `activate` script in every new shell before running playbooks.

# 2.3 Target-Node Requirements

# Target-Node Requirements

Every future cluster machine — control-plane, etcd, worker — must satisfy:

| Requirement | Value (per docs) |
|---|---|
| Supported OS | Ubuntu, Debian, RHEL/Rocky/Alma, openSUSE, Flatcar, Fedora/CoreOS, Amazon Linux 2, Oracle Linux, Kylin, openEuler… |
| Memory — control plane | **2 GB** or more |
| Memory — worker node | **1 GB** or more |
| CPU — control plane | **2 CPUs** or more |
| Swap | **Disabled** |
| IPv4 forwarding | **Enabled** ( `net.ipv4.ip_forward` ); IPv6 too if used |
| Network | Full connectivity + Internet/proxy to **pull images** |
| Unique IDs | hostname, MAC, product_uuid |

# Unique Node Identifiers

Every node must have a **unique** hostname, MAC address and **product_uuid** — cloned VMs often share these, which **breaks node registration**.

```
ip link                                  # MAC address per interface
sudo cat /sys/class/dmi/id/product_uuid  # product UUID
hostnamectl                              # hostname
```

- Two nodes with the same product_uuid or MAC may be treated as **one** node.
- Always check this **after cloning** a VM template — regenerate as needed.

> Kubespray keys the inventory on `inventory_hostname`; duplicate machine identities still cause kubelet registration conflicts at the cluster level.

# Disabling Swap & IP Forwarding

Kubernetes requires **swap off**; the kubelet refuses to start otherwise.
Kubespray turns it off, but you can do it by hand:

```
sudo swapoff -a
# Comment any swap line in /etc/fstab so it stays off after reboot
```

Enable IPv4 forwarding on every target (also set in the sysctls below):

```
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-k8s.conf
sudo sysctl --system
```

> Pod and Service traffic is **routed** through each node, so the kernel must be allowed to forward packets — enable IPv6 forwarding too if you run dual-stack.

# 2.4 Kernel: Version, Modules & sysctls

# Kernel Version Requirements

| Item | Detail |
|---|---|
| Kubernetes ≥ 1.32.0 | Recommended LTS kernel **4.19** (4.x); any **5.x / 6.x** also fine |
| cgroups v2 | Minimum kernel **4.15**, recommended **5.8+** |
| Old EL / Amazon | RHEL 8 / Alma 8 / Rocky 8 / Oracle 8 / Amazon Linux 2 ship kernels **below 4.19** |
| Below requirement? | Bypass the preflight check (group_vars): |

```
kubeadm_ignore_preflight_errors:
  - SystemVerification
```

> Prefer a kernel that **meets** the requirement; only use the ignore-errors escape hatch for stuck-on-old-EL hosts you cannot upgrade.

# Kernel Modules & sysctls

Two modules must be loaded so the bridge passes traffic through iptables:

| Module | Why |
|---|---|
| `overlay` | OverlayFS storage driver for the container runtime |
| `br_netfilter` | Bridge traffic visible to iptables (Service/CNI rules) |

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay && sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

> Kubespray's preinstall/runtime roles load these modules and apply these sysctls **for you** — a manual kubeadm host needs them set by hand.

# 2.5 Required Network Ports

# Required Ports — Control Plane

These ports must be **reachable** on each control-plane / etcd node:

| Proto | Port | Component |
|---|---|---|
| TCP | 22 | SSH (Ansible) |
| TCP | 6443 | Kubernetes API server |
| TCP | 2379 | etcd client API |
| TCP | 2380 | etcd peer API |
| TCP | 10250 | kubelet API |
| TCP | 10257 | kube-controller-manager |
| TCP | 10259 | kube-scheduler |

```
nc 127.0.0.1 6443 -zv -w 2     # test reachability of a port
```

> etcd uses **two** ports — **2379** for clients (API server), **2380** for peer-to-peer replication between etcd members.

# Required Ports — Workers

These ports must be reachable on each worker node:

| Proto | Port | Component |
|---|---|---|
| TCP | 22 | SSH (Ansible) |
| TCP | 10250 | kubelet API |
| TCP | 10256 | kube-proxy |
| TCP/UDP | 30000–32767 | NodePort service range |

- **10250** (kubelet) is on **every** node — control-plane and worker alike.
- The **30000–32767** range is where `type: NodePort` Services are exposed.

> Workers do **not** open the API/etcd/controller/scheduler ports — those live only on control-plane nodes.

# Required Ports — CNI-specific

Open these **only** for the CNI you actually deploy:

| CNI | Proto | Port | Purpose |
|---|---|---|---|
| Calico | TCP | 179 | BGP |
| Calico | UDP | 4789 | VXLAN |
| Calico | TCP | 5473 | Typha |
| Calico | UDP | 51820 / 51821 | WireGuard (v4 / v6) |
| Cilium | UDP | 8472 | VXLAN overlay |
| Cilium | TCP | 4240 | health checks |
| Cilium | TCP | 4244 / 4245 | Hubble server / relay |
| Cilium | UDP | 51871 | WireGuard |

> The CNI is chosen later in the inventory (Lesson 3+). Note its ports now so the firewall is ready before the network add-on rolls out.

# 2.6 Firewall & Putting It Together

# Kubespray Does Not Manage Your Firewall

> *"The firewalls are not managed — you'll need to implement your own rules."*

- Kubespray automates **swap off**, **kernel modules**, **sysctls** and the **bootstrap-os** Python install — but it will **not** open ports for you.
- You must open, before deploying: **6443** (API), **2379/2380** (etcd), **10250** (kubelet), **10257/10259** (controller/scheduler), **10256** + **30000–32767** (kube-proxy / NodePort), plus the **CNI** ports.
- A blocked port shows up later as nodes failing to join or Pods that can't reach the API — far harder to debug than opening them up front.

> Everything in this lesson is the **pre-flight checklist**. With the control node, target nodes, kernel and ports ready, the next lessons build the **inventory** and **run the install**.

# Key Takeaways

- **Two sides**: the **control node** runs Ansible (not in the cluster); the **target nodes** become control-plane / etcd / workers
- **Control node** = Python **3.11–3.13**, **ansible-core 2.18.x**, **netaddr**, in a **venv** via `pip install -r requirements.txt`
- **Target nodes**: supported OS, **2 GB**+ RAM + **2 CPU** control-plane / **1 GB**+ worker, **swap off**, **IPv4/IPv6 forwarding on**, kernel **≥ 4.19**
- Load `overlay` + `br_netfilter`; set **bridge-nf-call** + **ip_forward** sysctls (Kubespray automates this)
- Ensure unique **hostname / MAC / product_uuid** per node
- **Open the firewall yourself**: 6443, 2379/2380, 10250, 10257/10259, 10256, 30000–32767, plus CNI ports — these are the prereqs **before running Kubespray**

# End of Lesson 2

Next: Lesson 3 — Installation & Inventory

`python3 -m venv` · `pip install -r requirements.txt` · `swapoff -a` · `modprobe overlay br_netfilter` · ports 6443 / 2379-2380 / 10250 / 30000-32767